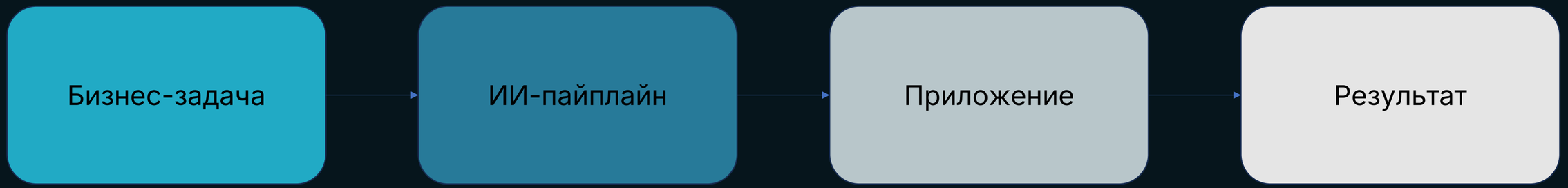
Агентский ИИ-пайплайн полного цикла разработки корпоративных приложений

От бизнес-требований до работающего приложения

Для кого:

- Системные аналитики;
- Продакт- и проджект-менеджеры;
- Аналитики данных;
- Разработчики ПО (фуллстек);
- ML-инженеры



Зачем нужен этот проект?

Команда **Russtech** регулярно получает запросы на создание внутренних цифровых сервисов:

- мобильных приложений;
- личных кабинетов;
- административных панелей;
- систем учета и мониторинга;
- инструментов автоматизации процессов.

Даже для задачи с готовыми бизнес-требованиями обычно нужна работа целой команды: аналитиков, дизайнеров, разработчиков, тестировщиков и специалистов по инфраструктуре и безопасности.

Проблематика: путь от описания задачи до первого работающего решения состоит из множества последовательных этапов.

1 этап

Прием и анализ требований

2 этап

Системный анализ

3 этап

Проектирование интерфейса

4 этап

Разработка ПО

5 этап

Тестирование и безопасность

6 этап

Сборка и развертывание

7 этап

Документация

Основной сценарий работы

Какую задачу мы решаем?

На данный момент

Бизнес-требования → множество ручных этапов → приложение

Выполнение каждого этапа сопряжено с необходимостью привлекать отдельных специалистов, согласовывать их расписание, проходить все процедуры разработки, в том числе создание артефактов

Хотим

Бизнес-требования → ИИ-пайплайн → готовый прототип

ИИ-агенты последовательно выполняют работу, передают результаты друг другу и проверяют итоговый результат. Сам по себе результат пригоден к развертыванию на пользователей и дальнейшей поддержке

Главная идея: автоматизировать не только написание кода, но и весь процесс создания прототипа, пригодного для тестирования бизнес-гипотезы — от анализа требований до запуска и документации.

Научить ИИ проходить весь путь разработки

Цель

Разработать процесс, в котором бизнес-требования будут трансформироваться в системный анализ, программный код, инфраструктуру, тесты, документацию и запущенный продукт.

Задача

Создать пригодный для практического применения ИИ-пайплайн разработки, который позволит ИИ-инженеру или другому бизнес-пользователю:

- самостоятельно выполнять последовательные этапы разработки;
- использовать результаты предыдущих этапов;
- формировать код и технические артефакты;
- запускать автоматизированные проверки;
- сохранять результаты в репозитории;
- минимизировать ручное вмешательство пользователя.

Что именно нужно разработать?

Участники создают не отдельное приложение для одного заранее известного сценария, **а универсальный конвейер.**

На вход пайплайна поступают исключительно бизнес-требования.
Способ взаимодействия пользователя с системой команда определяет самостоятельно.

Это может быть:

- Командная строка
- Веб-интерфейс
- Голосовой ввод
- Мобильный интерфейс
- Браузерный плагин
- Расширение для среды разработки

Технологический стек

Компоненты и дизайн-система web-интерфейса	UI Kit на выбор команды
Frontend	Angular 20.1.6
Backend	C# 14 / ASP.NET 10 Core
Data	PostgreSQL 17.9 (Debian 17.9-1.pgdg12+1)
Geo Data	PostGIS
Нативное Android-приложение	Kotlin 2.2.21
Обмен событиями	Kafka, REST API
Контейнеризация и локальный запуск	Docker / Docker Compose
Сборка, тестирование, проверки и развертывание	GitLab CI/CD 19.4.2
Генерация артефактов	LLM Qwen3.8-27B

Формат участия

1

Полноценный пайплайн

Реализовать всю последовательность от бизнес-требований до работающего и развернутого приложения

2

Специализированный участок

– анализ бизнес-требований;
– системный анализ;
– проектирование интерфейсов;
– генерация frontend;
– генерация backend;
– проектирование базы данных;
– тестирование;
– анализ безопасности;
– GitLab CI/CD;
– документирование.

Отсутствующие этапы не дисквалифицируют команду, но баллы за них не начисляются. Победить сможет как команда с наиболее полным решением, так и команда, создавшая выдающийся специализированный компонент.

Ожидаемый результат

Команда должна предоставить:

1. Работающий пайплайн или его значимую часть;
2. Репозиторий в локальном GitLab;
3. Сформированные документы;
4. Исходный код компонентов;
5. Настроенные тесты и проверки;
6. Docker-окружение;
7. CI/CD-конфигурацию;
8. Работающий прототип;
9. Пользовательскую документацию;
10. Отчет о расходе токенов, времени и количестве попыток;
11. Демонстрация видеопрезентации работы команды

Решения команд будут проходить проверку на двух проверочных кейсах:

- **Кейс № 1** — предоставляется командам для предварительной подготовки;
- **Кейс № 2** — остается нераскрытым контрольным заданием до начала итоговой проверки. Запуск и сборка проекта по этому кейсу выполняются строго в момент проведения оценки решений.

Как оцениваем?

1. Качество анализа

Максимум 20 баллов

2. Качество продукта и интерфейса

Максимум 15 баллов

3. Архитектура и разработка

Максимум 20 баллов

4. Тестирование и безопасность

Максимум 15 баллов

Как оцениваем?

5. Инфраструктура

Максимум 10 баллов

6. Документация

Максимум 10 баллов

7. Эффективность пайплайна

Максимум 10 баллов

Проверочный кейс: Парковочный помощник для технических служб

Контекст и проблематика

Сотрудники наших технических служб ежедневно выполняют работы на нескольких объектах Санкт-Петербурга, используя служебные автомобили. В течение рабочего дня водитель может перемещаться между объектами, расположенными в пределах одной или нескольких зон платной парковки.

Сейчас водитель самостоятельно контролирует необходимость оплаты, время окончания оплаченного периода и переход между парковочными зонами. Это увеличивает вероятность повторной оплаты, пропуска окончания сессии и получения штрафа.

Цель проверочного кейса

Создать прототип мобильного помощника, который определяет нахождение автомобиля в зоне платной парковки, отслеживает активную парковочную сессию и помогает водителю понять, требуется ли новая оплата.

Этот кейс используется для проверки способности пайплайна преобразовать бизнес-требования в системный анализ, интерфейс, программные компоненты, тесты, документацию и работающий прототип.

Функциональные требования

Система должна:

- определять местоположение автомобиля;
- определять нахождение автомобиля в платной парковочной зоне;
- показывать номер и границы зоны;
- отображать доступные параметры парковки;
- хранить информацию об активной парковочной сессии;
- отслеживать перемещение автомобиля;
- определять выход из текущей зоны;
- определять переход в другую парковочную зону;
- контролировать время окончания оплаченной сессии;
- предупреждать о необходимости проверки или новой оплаты;
- предоставлять переход к официальному сервису оплаты;
- отображать маршрут и историю парковочных сессий.

Примеры уведомлений:

Вы въехали в зону платной парковки

Парковка оплачена до 15:20

До окончания парковки осталось 10 минут

Вы покинули текущую парковочную зону

Вы въехали в другую парковочную зону. Проверьте необходимость новой оплаты

Максимум 10 баллов

Ожидаемые компоненты

Для проверочного кейса должны быть сформированы:

- Мобильное приложение на Kotlin
- PostgreSQL/PostGIS
- Docker-окружение
- CI/CD-пайплайн
- Административный web-интерфейс на Angular
- Документированный API
- Тестовые данные
- Автоматизированные тесты
- Backend на C# / ASP.NET Core
- Модель событий
- Системная, продуктовая и пользовательская документация

Источники данных и интеграции

В проверочном сценарии могут использоваться:

- Открытые данные о парковочных зонах Санкт-Петербурга
- GPS-координаты или демонстрационный mock-маршрут
- Геоданные парковочных зон
- Официальный сервис парковок Санкт-Петербурга
- Приложение «Парковки Санкт-Петербурга» или «Парковки России»
- Сервис и API «Паркоматика»

Получение промышленного доступа и выполнение реальной оплаты не требуются. Интеграция должна быть реализована через адаптер и mock-сервис, чтобы внешнего поставщика можно было заменить без изменения основной бизнес-логики. Собственный платежный механизм разрабатывать не требуется.

Демонстрационный сценарий

Во время демонстрации система должна обработать последовательность событий:

Служебный автомобиль движется по заданному маршруту

Автомобиль въезжает в зону платной парковки

Система определяет зону и уведомляет водителя

Водитель запускает или подтверждает парковочную сессию

Система показывает время ее окончания

За 10 минут до окончания поступает предупреждение

Автомобиль покидает зону или перемещается в другую зону

Система определяет изменение и предлагает проверить необходимость новой оплаты

События отображаются в административном интерфейсе и сохраняются в истории

Агентский ИИ-пайплайн полного цикла разработки корпоративных приложений